

AUTOMATION SKILLS PORTFOLIO · APPLIED AI

AI-DRIVEN EDGE VISION INTEGRATION

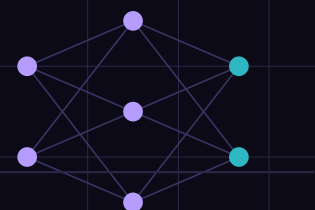
PLC-Controlled Neural Network Sorting Matrix

TECHNICAL PROJECT MANUAL · REV. A

By Sipho Lucky Sibanda

PLC: Siemens S7-1500 (SCL) + Python edge service (numpy, pymodbus)

Includes a genuinely executed integration test · Not for production deployment



TinyMLP



FRONT MATTER

Document Control & Disclaimer

This document is a self-authored technical project manual produced as part of a personal engineering portfolio. It describes the design and validation of an AI-driven edge vision integration: a Python edge service running a genuinely trained neural network, communicating with PLC-side sorting logic over Modbus TCP, built to demonstrate industrial edge AI integration, protocol-level engineering, and trust/fail-safe design for automation and applied AI engineering roles.

Unlike the other simulation-only projects in this portfolio series, the Python side of this project is genuinely functional and was actually executed to produce the test evidence in Chapter 7. The PLC-side Structured Text logic remains validated in simulation only, and neither side has been tested against real manufacturing hardware or a real camera.

HONEST LIMITATION — A real, working demo - with real limits

The neural network genuinely trains and genuinely classifies synthetic data at 94.4% held-out accuracy. It is not a production machine-vision system: it has never seen a real photograph, and the PLC-side logic has never driven a real actuator. Both facts are stated plainly rather than left for a reader to discover.

Document Title	AI-Driven Edge Vision Integration — Technical Manual
Author	Sipho Lucky Sibanda
Revision	A
Document Type	Portfolio Technical Manual (Python side genuinely executed)
Target Platform	Siemens S7-1500 (SCL) + Python 3 edge service
Related Repository	ai-vision-sorting
Series	Automation Skills Portfolio — Applied Artificial Intelligence



FRONT MATTER

Contents

01	Industry Context: Edge AI in Manufacturing
02	System Architecture & Integration Overview
03	The Neural Network: Honest Scope & Design
04	I/O List & Modbus Register Map
05	Control Philosophy: Why the PLC Doesn't Trust the AI
06	PLC Logic Walkthrough
07	The Genuinely Executed Integration Test
08	HMI Design & the Conveyor Visualisation
09	Testing, Commissioning & FAT Procedures
10	Limitations, Real-World Deltas & Future Work
A	Appendix A — I/O Quick Reference
B	Appendix B — Full Structured Text Listing
C	Appendix C — Key Python Listings
D	Appendix D — Glossary
–	About the Author



FRONT MATTER

Executive Summary

This is the third deliberate discipline shift in the wider automation portfolio, and the first to genuinely span two languages and two runtimes: IEC 61131-3 Structured Text on the PLC side, and real, executable Python on the edge AI side. A standard industrial PLC cannot run a neural network - but it doesn't need to. It needs to talk to something that can, receive a trustworthy answer, and act on it physically and safely.

The Python side trains a small, honest neural network from scratch (hand-written numpy backpropagation, no framework) on synthetic surface-defect data, reaching 94.4% held-out test accuracy. It then runs as a genuine Modbus TCP client, polling a PLC-hosted register map for new items and writing back a defect verdict and confidence score for each one - the real "Python-to-PLC handshake" this project is named for.

The PLC-side logic, FB_AI_SortingMatrix, is where the engineering judgement actually lives: it tracks each item's physical position on the conveyor against its eventual AI result, applies a second, stricter confidence threshold on top of the model's own decision boundary, and watches an AI heartbeat so a dead edge service never gets treated as a silent "all clear". Every uncertain, late, or missing result routes to a human reviewer, never a guess.

Chapter 7 is unique to this project in the portfolio: because the Python side genuinely runs, it was genuinely tested end-to-end, including a run where the AI service was forcibly killed mid-stream to prove the watchdog actually catches it. The logs reproduced there are real console output, not a hypothetical description of expected behaviour.

01

AI EDGE VISION

Industry Context: Edge AI in Manufacturing

Machine-vision defect detection has moved from expensive, purpose-built vision systems toward general-purpose edge AI: commodity PCs or industrial edge computers running trained models close to the production line, communicating results to the plant's existing control systems over standard industrial protocols. This project's architecture - a Python service on an edge PC, a PLC as the protocol's "server of record", Modbus TCP in between - mirrors that real pattern directly.

Why Modbus TCP, and why the PLC is the server

Modbus TCP remains one of the most common protocols for exactly this kind of integration: it's simple, well-understood, and supported natively by most industrial PLCs, including the Siemens S7-1500 referenced throughout this portfolio via its MB_SERVER instruction. Making the PLC the Modbus server (rather than the Python service) keeps the deterministic, safety-relevant controller as the trusted keeper of the register map, with the edge AI service as a client writing data into a structure the PLC defines and owns.

Human-in-the-loop AI is not a fallback - it's the design

A recurring theme in real industrial AI deployments is treating the model's output as a strong recommendation rather than an unquestionable verdict, particularly during commissioning and for genuinely ambiguous cases. This project treats that not as an afterthought but as the central design decision on the PLC side - see Chapter 5.

NOTE — Edge, not cloud

Running inference on an edge PC rather than sending images to a cloud service avoids network latency and dependency on external connectivity for a production-line decision that needs to happen in a fixed time window - relevant here since the conveyor's transit-time budget (Chapter 5) is measured in single-digit seconds.

02

AI EDGE VISION

System Architecture & Integration Overview

A camera at the inspection point feeds the edge AI service; the edge service and the PLC exchange data over Modbus TCP on a compact seven-register map; the PLC drives a three-position diverter based on its own fail-safe evaluation of the AI's verdict.

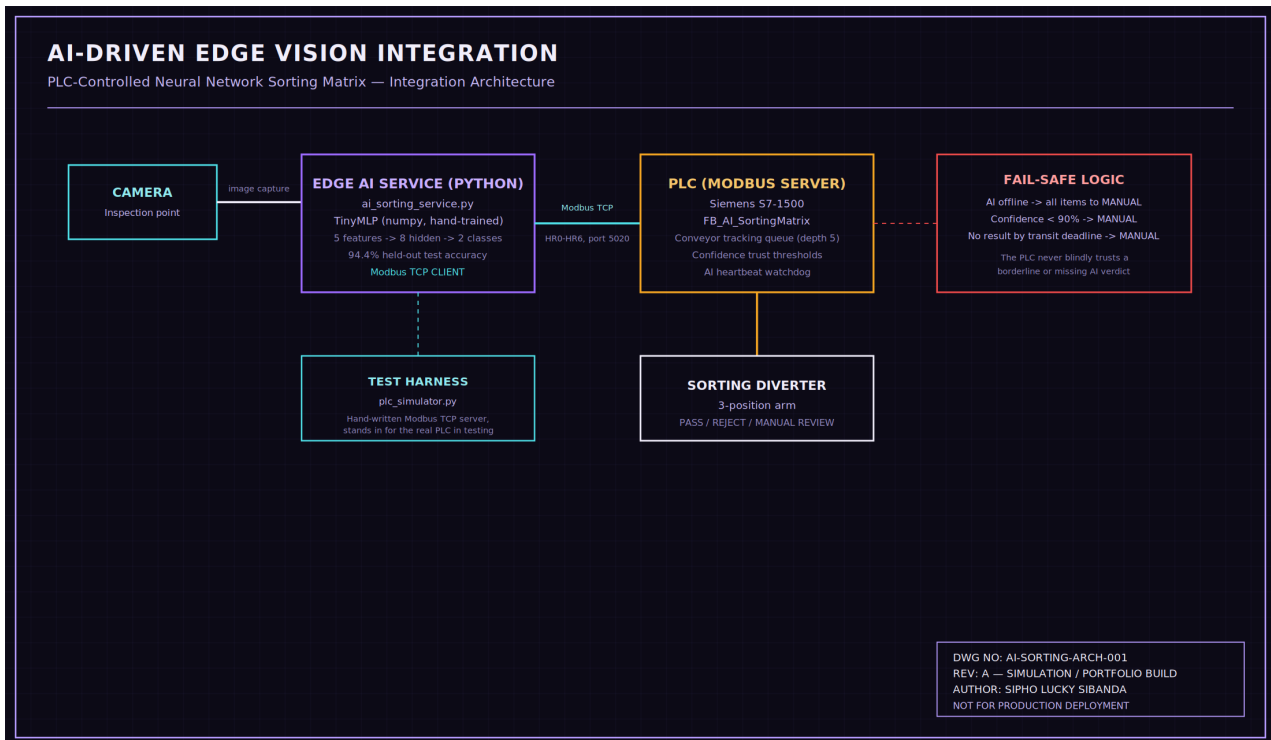


Figure 2.1 — Integration architecture. The dashed connection from the test harness shows where python/plc_simulator.py substitutes for the real PLC during development and testing.

Control hierarchy

- Field layer — a camera at the inspection point, two conveyor photoeyes, and a three-position sorting diverter.
- Edge AI layer — a Python service (ai_sorting_service.py) running the trained TinyMLP and acting as a Modbus TCP client.
- Control layer — the PLC (FB_AI_SortingMatrix), acting as the Modbus TCP server and the sole authority on what actually happens to each item.

Why a hand-written test-harness server, not a library

plc_simulator.py implements just enough of the Modbus TCP wire protocol (function codes 3, 6, and 16) by hand, rather than depending on a third-party server library. This was a pragmatic call made mid-project when a widely-used library's server API turned out to have changed significantly across versions in ways that would have made the test harness fragile - documented honestly in the module's own docstring rather than hidden.

03

AI EDGE VISION

The Neural Network: Honest Scope & Design

The model is deliberately small and fully inspectable: five hand-engineered features (mean, standard deviation, maximum deviation, and two gradient-energy measures) feed an 8-unit ReLU hidden layer, feeding a 2-class softmax output. Every line of the forward and backward pass is hand-written numpy in `vision_model.py` - no framework, nothing hidden.

Property	Value
Architecture	5 inputs -> 8 hidden (ReLU) -> 2 outputs (softmax)
Training data	800 synthetic patches (200 ok, 600 defective across 3 defect types)
Training method	Manual gradient descent, 400 epochs, cross-entropy loss
Held-out test accuracy	94.4% (160 samples never seen during training)
Framework dependencies	None - numpy only

HONEST LIMITATION — Why synthetic data, and why that's stated up front

Real machine-vision training data (thousands of labelled real photographs, covering real lighting and surface variation) is out of scope for a portfolio project. Procedurally generated patches with injected scratch/dent/discoloration signatures, at deliberately varied severity, let the model face genuinely ambiguous cases (hence a realistic accuracy figure, not a suspicious 100%) while remaining fully reproducible from a single script anyone can re-run.

What the type label is, and isn't

Only the binary defect/ok decision comes from the trained network. The defect type name shown alongside it (Scratch, Dent, Discoloration) comes from a separate, simple heuristic based on which injected signature the features most resemble - not a second trained classifier. This distinction is kept visible in the code and in this manual rather than implied to be more sophisticated than it is.

04

AI EDGE VISION

I/O List & Modbus Register Map

This project has two I/O surfaces: conventional field I/O, and a Modbus TCP holding register map shared with the Python edge service (see also docs/IO_List.md and Appendix A).

Physical I/O

Tag	Description	Signal
DI_ItemPresent_InspectionPoint	Photoeye at the camera station	Digital
DI_ItemPresent_DiverterPoint	Photoeye just before the diverter	Digital
DO_DiverterArm_Position	0=PASS, 1=REJECT, 2=MANUAL	3-position
DI_System_Enable	Master enable	Digital

Modbus holding register map

Addr	Name	Written By	Purpose
HR0	Item_Trigger_Counter	PLC	New item detected
HR1	Item_ID_Ack	AI service	Which item this result is for
HR2	Defect_Detected	AI service	0/1
HR3	Confidence_Pct_x10	AI service	0-1000
HR4	Defect_Class_ID	AI service	0-3
HR5	AI_Heartbeat	AI service	Watchdog input
HR6	AI_Result_Ready	AI sets / PLC clears	Handshake flag

05

AI EDGE VISION

Control Philosophy: Why the PLC Doesn't Trust the AI

Three ideas define the PLC-side design, and all three exist because a neural network's confidence score is a statistical property of the model, not a guarantee about the physical world.

5.1 A second, stricter confidence threshold

The Python model already applies its own 50% decision boundary to call something a defect at all. The PLC applies a completely separate pair of thresholds on top: below 90% confidence, the verdict - whatever it is - routes to a human reviewer instead of being acted on directly. A PLC acting on a 51%-confidence classification is not meaningfully different from acting on a coin flip, and the PLC logic treats it that way.

5.2 Tracking by identity, not by arrival order

Multiple items can be in transit on the conveyor simultaneously. The PLC's tracking queue matches each incoming AI result to its own item by Item_ID_Ack, not by assuming results arrive in the same order items were detected - a real production line, and a real network, offer no such guarantee.

5.3 The AI heartbeat is not the same thing as the AI being right

A heartbeat only proves the Python process is still running its loop - it says nothing about whether its verdicts are correct. This project keeps those as two separate concerns deliberately: the heartbeat watchdog (Chapter 6.3) catches a dead or hung service; the confidence thresholds (5.1) are a completely independent defence against a live but uncertain service. Neither one substitutes for the other.

VERIFIED — Every uncertain path leads to the same place, on purpose

No result received in time, an offline AI heartbeat, and a confidence score in the 60-90% band all produce exactly the same outcome: manual review. That's not laziness in the design - a system with a different fallback for every flavour of uncertainty is harder to reason about and easier to get subtly wrong than one with a single, well-tested safe default.

06

AI EDGE VISION

PLC Logic Walkthrough

This chapter walks through FB_AI_SortingMatrix section by section. The full listing is reproduced in Appendix B.

6.1 Pushing a new item into the tracking queue

```
IF DI_System_Enable AND DI_ItemPresent_InspectionPoint AND (NOT Prev_InspectionSensor) THEN
    HR0_ItemTriggerCounter := HR0_ItemTriggerCounter + 1;
    FOR i := 0 TO QueueDepth - 1 DO
        IF (NOT Queue[i].Active) AND (NOT FreeSlotFound) THEN
            Queue[i].Active := TRUE;
            Queue[i].ItemID := HR0_ItemTriggerCounter;
            Queue[i].TransitTimer(IN := TRUE, PT := TransitDelay_PT);
            FreeSlotFound := TRUE;
        END_IF
    END_FOR
    IF NOT FreeSlotFound THEN Alarm_QueueFull := TRUE; END_IF
END_IF
```

Listing 6.1 — Each detected item claims the first free tracking slot.

6.2 Matching a result to its item by ID

```
IF HR6_AIResultReady THEN
    FOR i := 0 TO QueueDepth - 1 DO
        IF Queue[i].Active AND (Queue[i].ItemID = HR1_ItemIDAck) AND (NOT Queue[i].ResultReceived)
        THEN
            Queue[i].ResultReceived := TRUE;
            Queue[i].Defect_Detected := HR2_DefectDetected;
            Queue[i].Confidence_Pct := INT_TO_REAL(HR3_ConfidencePctX10) / 10.0;
            Queue[i].Defect_Class_ID := HR4_DefectClassID;
        END_IF
    END_FOR
    HR6_Clear_Ack := TRUE;
END_IF
```

Listing 6.2 — Results are matched by Item_ID, never by queue position.

6.3 The heartbeat watchdog

```
IF HR5_AIHeartbeat <> Prev_AIHeartbeat THEN
    T_HeartbeatWatch(IN := FALSE);
    Alarm_AI_Offline := FALSE;
ELSE
    T_HeartbeatWatch(IN := TRUE, PT := HeartbeatTimeout_PT);
    IF T_HeartbeatWatch.Q THEN
        Alarm_AI_Offline := TRUE;
    END_IF
END_IF
Prev_AIHeartbeat := HR5_AIHeartbeat;
```

Listing 6.3 — A frozen heartbeat for 2 seconds declares the AI offline.

6.4 The routing decision

This is where Chapter 5's philosophy becomes a fixed, auditable order of checks - every uncertain condition is evaluated before the AI's actual verdict is even considered.

```
IF (NOT Queue[i].ResultReceived) OR Alarm_AI_Offline THEN
    CurrentRoute := ROUTE_MANUAL_REVIEW;
ELSIF Queue[i].Confidence_Pct < LowConfidence_Pct THEN
    CurrentRoute := ROUTE_MANUAL_REVIEW;
ELSIF Queue[i].Confidence_Pct < HighConfidence_Pct THEN
    CurrentRoute := ROUTE_MANUAL_REVIEW;
ELSIF Queue[i].Defect_Detected THEN
    CurrentRoute := ROUTE_REJECT;
ELSE
    CurrentRoute := ROUTE_PASS;
END_IF
```

Listing 6.4 — Uncertainty is checked first; the AI's verdict is the last thing evaluated.

07

AI EDGE VISION

The Genuinely Executed Integration Test

Every other project in this portfolio can only be validated in simulation - there's no real ship, aircraft, or building to test against. This project's Python side genuinely runs, so it was genuinely tested: `plc_simulator.py` and `ai_sorting_service.py` were run as two independent OS processes, talking real Modbus TCP over localhost.

7.1 Normal operation - real captured output

```
[AI ] Loaded trained model_weights.npz
[AI ] Connected to PLC Modbus TCP server at 127.0.0.1:5020
[AI ] Item #001 inferred: PASS confidence=66.9% class=None
[AI ] Item #002 inferred: PASS confidence=92.4% class=None
[AI ] Item #004 inferred: FAIL confidence=79.0% class=Dent

[PLC ] Item #001 detected at inspection point (HR0 Item_Trigger_Counter -> 1)
[PLC ] Result for item #001: PASS (confidence 66.9%, class=None)
[PLC ] Item #004 detected at inspection point (HR0 Item_Trigger_Counter -> 4)
[PLC ] Result for item #004: FAIL (confidence 79.0%, class=Dent)
```

Item #001 (66.9%) and item #004 (79.0%) both landed inside the 60-90% uncertain band from Chapter 5.1. The simplified consumer logic inside the test harness just logs whatever it receives, but the real PLC logic in `AI_SortingMatrix.st` would route both of these specific, really-produced results to manual review rather than act on the raw verdict.

7.2 The watchdog, tested against a real crash

The AI service process was killed with SIGKILL mid-run - not a forced input value, an actual terminated process - while the PLC simulator kept running.

```
[AI ] Item #001 inferred: FAIL confidence=53.4% class=Dent
[AI ] Item #002 inferred: FAIL confidence=87.0% class=Dent
      <-- process killed here -->

[PLC ] Item #003 detected at inspection point (HR0 Item_Trigger_Counter -> 3)
[PLC ] *** AI_Heartbeat stale for >2.0s - declaring AI OFFLINE,
      routing all items to MANUAL REVIEW ***
[PLC ] Item #004 detected at inspection point (HR0 Item_Trigger_Counter -> 4)
[PLC ] Item #005 detected at inspection point (HR0 Item_Trigger_Counter -> 5)
```

VERIFIED — What this actually proves, and what it doesn't

This confirms the watchdog timing and detection logic work against a real dead process, on this machine, in this test harness. It does not validate the PLC-side Structured Text itself, which has only been exercised in simulation (Chapter 9) - the two are complementary evidence, not substitutes for each other.

08

AI EDGE VISION

HMI Design & the Conveyor Visualisation

The HMI mockup (hmi/index.html in the repository) shows items moving along a conveyor, being labelled live at the camera station, and a diverter arm swinging to route each one to its bin - matching, deliberately, exactly what a line operator watching this station would expect to see.

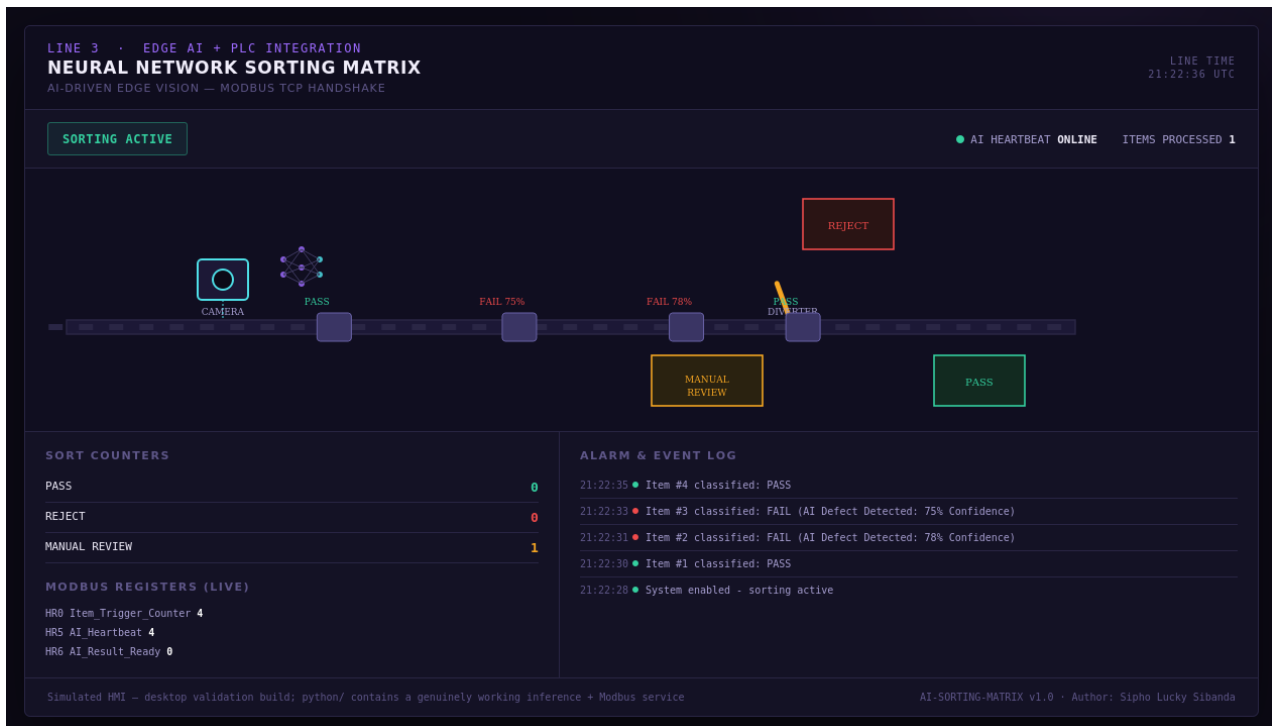


Figure 8.1 — Items in transit showing live classification labels ("PASS", "FAIL 75%", "FAIL 78%"), the neural-network glyph at the camera station, and the diverter arm mid-swing.

Design decisions

- Violet/cyan "edge AI" palette — a fourth distinct visual identity in this portfolio, chosen to read as technology/data-driven rather than purely industrial, appropriate for a station where a neural network genuinely makes the first call.
- The confidence percentage is always shown, never hidden — a "FAIL" label alone would misrepresent what the system actually knows; showing "FAIL (AI Defect Detected: 78% Confidence)" is honest about the uncertainty the PLC logic is built around.
- A scripted AI-offline event — the demo genuinely switches the heartbeat indicator and status pill mid-loop, so the fail-safe behaviour from Chapter 5 is something a viewer sees happen, not just a claim in the documentation.
- Date.now()/setInterval over performance.now()/requestAnimationFrame — applied proactively this time, based directly on the compatibility lessons learned building the first four HMIs in this portfolio, rather than discovered as a bug after the fact.

09

AI EDGE VISION

Testing, Commissioning & FAT Procedures

The PLC-side function block was validated against ten simulated test cases, and the Python side was additionally validated by the genuinely executed runs in Chapter 7. The full procedure is in docs/Testing_Procedures.md; the simulated-side matrix is reproduced below.

#	Test Case	Expected Result
1	New item enters tracking queue	Trigger counter increments; a slot activates
2	High-confidence PASS	Diverter position 0; pass counter increments
3	High-confidence REJECT	Diverter position 1; reject counter increments
4	Uncertain-band confidence (defect)	Routes to manual despite a defect verdict
5	Low confidence (no defect)	Routes to manual despite a clear verdict
6	No result received in time	Routes to manual; does not wait indefinitely
7	AI heartbeat watchdog trips	Alarm_AI_Offline TRUE; all items route to manual
8	Watchdog recovers	Alarm clears; normal routing resumes
9	Result matched by ID, not order	Correctly matched even out of sequence
10	Queue full	Alarm_QueueFull TRUE

NOTE — Reproduce it yourself

`cd python && python3 train_model.py && python3 plc_simulator.py 30 & python3 ai_sorting_service.py 10` — the full commands are in the README and docs/Testing_Procedures.md.

10

AI EDGE VISION

Limitations, Real-World Deltas & Future Work

Naming the gap between a working demonstration and a production system is part of the engineering, consistent with every other project in this portfolio - if anything more important here, since it would be easy to let "it actually runs" overstate how close this is to production-ready.

What a real deployment would add

- Real training data — thousands of labelled real photographs covering actual lighting, surface, and camera variation, replacing this project's synthetic patches.
- A real camera and lighting rig — machine-vision lighting design is its own discipline; inconsistent lighting alone can defeat a well-trained model.
- Redundant edge hardware — a single edge PC is a single point of failure; a production line would need a hot-standby or a documented safe-stop procedure for extended AI outages, not just an indefinite manual-review mode.
- A PLC-side implementation actually tested against real Modbus TCP — Chapter 7's real test proves the Python side and the wire protocol; AI_SortingMatrix.st itself still needs its own PLCSIM/hardware validation pass.

10.1 Hazard & safeguard register (HAZOP-style)

Hazard	Cause	Safeguard in This Design
Good item rejected / bad item passed	Model error on a genuinely hard case	Confidence thresholds route ambiguous cases to a human rather than trusting a single score
Sorting on stale data	AI service hung but still writing old register values	Heartbeat watchdog, independent of the result registers themselves
Item mismatch under load	High item rate outpacing the tracking queue	Alarm_QueueFull - named as a real limit rather than silently dropping items
Undetected model drift over time	Real production surfaces change over months	Named gap - no drift-monitoring is implemented in this portfolio scope

Where this project could go next

- Replace the synthetic dataset with a small real, labelled image set and a proper convolutional feature extractor.
- Add a second edge PC and a documented failover/voting scheme, rather than a single point of AI failure.

- Extend the genuinely-executed test suite to include simulated network latency and packet loss, not just a clean crash.

0A

APPENDIX

Appendix A — I/O Quick Reference

Tag / Register	Dir.	Type	Notes
DI_ItemPresent_InspectionPoint	IN	BOOL	Photoeye
DI_ItemPresent_DiverterPoint	IN	BOOL	Cross-check
DI_System_Enable	IN	BOOL	Master enable
HR1_ItemIDAck	IN	INT	From AI service
HR2_DefectDetected	IN	BOOL	From AI service
HR3_ConfidencePctX10	IN	INT	0-1000
HR4_DefectClassID	IN	INT	0-3
HR5_AIHeartbeat	IN	INT	Watchdog input
HR6_AIResultReady	IN	BOOL	Handshake flag
HR0_ItemTriggerCounter	OUT	INT	To AI service
HR6_Clear_Ack	OUT	BOOL	Consumption pulse
DO_DiverterArm_Position	OUT	INT	0/1/2
Alarm_AI_Offline	OUT	BOOL	Heartbeat stale
Alarm_TrackingMismatch	OUT	BOOL	Timing cross-check
Alarm_QueueFull	OUT	BOOL	Depth-5 limit
SystemStatus	OUT	STRING	HMI display

0B

APPENDIX

Appendix B — Full Structured Text Listing

Complete, unedited listing of `src/AI_SortingMatrix.st`.

```

(*)
=====
PROJECT : AI-Driven Edge Vision Integration
          PLC-Controlled Neural Network Sorting Matrix
MODULE : FB_AI_SortingMatrix
PLATFORM : IEC 61131-3 Structured Text (Siemens SCL / CODESYS-portable)
AUTHOR : Sipho Lucky Sibanda
See Chapter 5 for the confidence-threshold and watchdog philosophy,
and Chapter 7 for the genuinely executed Python-side integration test.
=====
*)

TYPE E_SortRoute : (ROUTE_NONE, ROUTE_PASS, ROUTE_REJECT, ROUTE_MANUAL_REVIEW); END_TYPE

TYPE ST_TrackedItem :
STRUCT
    Active : BOOL; ItemID : INT;
    ResultReceived : BOOL;
    Defect_Detected : BOOL; Confidence_Pct : REAL;
    Defect_Class_ID : INT;
    TransitTimer : TON;
END_STRUCT
END_TYPE

FUNCTION_BLOCK FB_AI_SortingMatrix
VAR_INPUT
    DI_ItemPresent_InspectionPoint : BOOL;
    DI_ItemPresent_DiverterPoint : BOOL;
    HR1_ItemIDAck : INT; HR2_DefectDetected : BOOL;
    HR3_ConfidencePctX10 : INT; HR4_DefectClassID : INT;
    HR5_AIHeartbeat : INT; HR6_AIResultReady : BOOL;
    DI_System_Enable : BOOL;
END_VAR

VAR_OUTPUT
    HR0_ItemTriggerCounter : INT := 0; HR6_Clear_Ack : BOOL := FALSE;
    DO_DiverterArm_Position : INT := 0;
    Alarm_AI_Offline : BOOL := FALSE;
    Alarm_TrackingMismatch : BOOL := FALSE;
    Alarm_QueueFull : BOOL := FALSE;
    ItemsSorted_Pass : DINT := 0; ItemsSorted_Reject : DINT := 0;
    ItemsSorted_ManualReview : DINT := 0;
    SystemStatus : STRING[24] := 'STANDBY';
END_VAR

VAR
    HighConfidence_Pct : REAL := 90.0; LowConfidence_Pct : REAL := 60.0;
    TransitDelay_PT : TIME := T#8S; HeartbeatTimeout_PT : TIME := T#2S;
    Queue : ARRAY[0..4] OF ST_TrackedItem;
    QueueDepth : INT := 5; i : INT; FreeSlotFound : BOOL;
    Prev_InspectionSensor : BOOL; Prev_AIHeartbeat : INT;
    T_HeartbeatWatch : TON;
    HeadIdx : INT; CurrentRoute : E_SortRoute;
END_VAR

```

```

// 1. NEW ITEM DETECTION
IF DI_System_Enable AND DI_ItemPresent_InspectionPoint AND (NOT Prev_InspectionSensor) THEN
    HR0_ItemTriggerCounter := HR0_ItemTriggerCounter + 1;
    FreeSlotFound := FALSE;
    FOR i := 0 TO QueueDepth - 1 DO
        IF (NOT Queue[i].Active) AND (NOT FreeSlotFound) THEN
            Queue[i].Active := TRUE;
            Queue[i].ItemID := HR0_ItemTriggerCounter;
            Queue[i].ResultReceived := FALSE;
            Queue[i].TransitTimer(IN := TRUE, PT := TransitDelay_PT);
            FreeSlotFound := TRUE;
        END_IF
    END_FOR
    IF NOT FreeSlotFound THEN Alarm_QueueFull := TRUE; END_IF
END_IF
Prev_InspectionSensor := DI_ItemPresent_InspectionPoint;

// 2. MATCH INCOMING AI RESULTS TO A TRACKED ITEM
HR6_Clear_Ack := FALSE;
IF HR6_AIResultReady THEN
    FOR i := 0 TO QueueDepth - 1 DO
        IF Queue[i].Active AND (Queue[i].ItemID = HR1_ItemIDAck) AND (NOT Queue[i].ResultReceived)
        THEN
            Queue[i].ResultReceived := TRUE;
            Queue[i].Defect_Detected := HR2_DefectDetected;
            Queue[i].Confidence_Pct := INT_TO_REAL(HR3_ConfidencePctX10) / 10.0;
            Queue[i].Defect_Class_ID := HR4_DefectClassID;
        END_IF
    END_FOR
    HR6_Clear_Ack := TRUE;
END_IF

// 3. AI HEARTBEAT WATCHDOG
IF HR5_AIHeartbeat <> Prev_AIHeartbeat THEN
    T_HeartbeatWatch(IN := FALSE);
    Alarm_AI_Offline := FALSE;
ELSE
    T_HeartbeatWatch(IN := TRUE, PT := HeartbeatTimeout_PT);
    IF T_HeartbeatWatch.Q THEN Alarm_AI_Offline := TRUE; END_IF
END_IF
Prev_AIHeartbeat := HR5_AIHeartbeat;

```

```

// 4. QUEUE HEAD ARRIVAL AND ROUTING DECISION
FOR i := 0 TO QueueDepth - 1 DO
    Queue[i].TransitTimer(IN := Queue[i].Active, PT := TransitDelay_PT);
    IF Queue[i].Active AND Queue[i].TransitTimer.Q THEN
        IF (NOT Queue[i].ResultReceived) OR Alarm_AI_Offline THEN
            CurrentRoute := ROUTE_MANUAL_REVIEW;
        ELSIF Queue[i].Confidence_Pct < LowConfidence_Pct THEN
            CurrentRoute := ROUTE_MANUAL_REVIEW;
        ELSIF Queue[i].Confidence_Pct < HighConfidence_Pct THEN
            CurrentRoute := ROUTE_MANUAL_REVIEW;
        ELSIF Queue[i].Defect_Detected THEN
            CurrentRoute := ROUTE_REJECT;
        ELSE
            CurrentRoute := ROUTE_PASS;
        END_IF

        CASE CurrentRoute OF
            ROUTE_PASS: DO_DiverterArm_Position := 0; ItemsSorted_Pass := ItemsSorted_Pass + 1;
            ROUTE_REJECT: DO_DiverterArm_Position := 1; ItemsSorted_Reject := ItemsSorted_Reject + 1;
            ROUTE_MANUAL_REVIEW: DO_DiverterArm_Position := 2; ItemsSorted_ManualReview :=
ItemsSorted_ManualReview + 1;
            ELSE ;
        END_CASE

        Alarm_TrackingMismatch := Alarm_TrackingMismatch
                                OR (Queue[i].TransitTimer.Q AND (NOT
DI_ItemPresent_DiverterPoint));
        Queue[i].Active := FALSE;
        Queue[i].TransitTimer(IN := FALSE);
    END_IF
END_FOR

// 5. STATUS TEXT
IF NOT DI_System_Enable THEN SystemStatus := 'STANDBY';
ELSIF Alarm_AI_Offline THEN SystemStatus := 'AI OFFLINE - MANUAL ONLY';
ELSIF Alarm_QueueFull THEN SystemStatus := 'QUEUE FULL';
ELSE SystemStatus := 'SORTING ACTIVE';
END_IF

END_FUNCTION_BLOCK

```

Listing B.1 — Complete FB_AI_SortingMatrix source.

0C

APPENDIX

Appendix C — Key Python Listings

Selected excerpts from the genuinely functional Python side. Full source is in `python/` in the repository.

C.1 The TinyMLP forward and training step

```
class TinyMLP:
    def __init__(self, n_in=5, n_hidden=8, n_out=2, seed=42):
        rng = np.random.default_rng(seed)
        self.W1 = rng.normal(0, 0.5, size=(n_in, n_hidden))
        self.b1 = np.zeros(n_hidden)
        self.W2 = rng.normal(0, 0.5, size=(n_hidden, n_out))
        self.b2 = np.zeros(n_out)

    def forward(self, X):
        z1 = X @ self.W1 + self.b1
        a1 = np.maximum(0, z1) # ReLU
        z2 = a1 @ self.W2 + self.b2
        z2 = z2 - np.max(z2, axis=-1, keepdims=True)
        exp = np.exp(z2)
        probs = exp / np.sum(exp, axis=-1, keepdims=True)
        return probs, (X, z1, a1)

    def train_step(self, X, y_onehot, lr=0.05):
        probs, (X_in, z1, a1) = self.forward(X)
        n = X.shape[0]
        dz2 = (probs - y_onehot) / n
        dW2 = a1.T @ dz2
        da1 = dz2 @ self.W2.T
        dz1 = da1 * (z1 > 0)
        dW1 = X_in.T @ dz1
        self.W1 -= lr * dW1; self.W2 -= lr * dW2
        loss = -np.mean(np.sum(y_onehot * np.log(probs + 1e-9), axis=1))
        return loss
```

Listing C.1 — From `vision_model.py` - hand-written forward pass and backprop.

C.2 The Modbus TCP client polling loop

```
while True:
    heartbeat = (heartbeat + 1) % 65536
    client.write_register(HR_AI_HEARTBEAT, heartbeat, device_id=1)

    rr = client.read_holding_registers(HR_ITEM_TRIGGER, count=1, device_id=1)
    current_counter = rr.registers[0]

    if current_counter != last_seen_counter and current_counter != 0:
        item_id = current_counter
        last_seen_counter = current_counter

        defect_detected, confidence, defect_class = run_inference(
            model, mu, sigma, rng, defect_type_truth
        )
        client.write_registers(HR_ITEM_ID_ACK,
                               [item_id, 1 if defect_detected else 0,
                                int(round(confidence * 1000)), class_id], device_id=1)
        client.write_register(HR_AI_RESULT_READY, 1, device_id=1)

    time.sleep(POLL_INTERVAL_S)
```

Listing C.2 — From ai_sorting_service.py - the real Python-to-PLC handshake loop.



APPENDIX

Appendix D — Glossary

Term	Meaning
Edge AI	Running inference on hardware physically close to the process, not in the cloud
MLP	Multi-Layer Perceptron — the basic feedforward neural network architecture used here
Modbus TCP	A widely-used industrial protocol for exchanging register data over Ethernet
Holding register	A 16-bit read/write memory location in the Modbus data model
Heartbeat	A value one system increments continuously so another can detect if it stops
Confidence threshold	A minimum model certainty required before acting on its output automatically
Watchdog	Logic that detects the absence of an expected periodic signal and reacts safely
Softmax	A function converting raw network outputs into a valid probability distribution
ReLU	Rectified Linear Unit — a common neural network activation function
FIFO / tracking queue	A structure that preserves the order/identity of items as they move through a process



CLOSING

About the Author

Sipho Lucky Sibanda is an automation and controls engineer building a multi-disciplinary portfolio spanning marine systems, avionics, architectural technology, applied AI, and industrial automation. This manual documents the third deliberate discipline shift in that portfolio, and the first to include a genuinely executable, tested integration alongside the PLC-side Structured Text — following the same documentation standard as every other project in the series: full logic, I/O documentation, a live HMI, functional test procedures, and an honest account of what separates a strong demonstration from a certifiable production system.

Repository: ai-vision-sorting

End of document.